

# Paid Interview at Array

Welcome to the Array Engineering Challenge - a two-day paid interview designed to evaluate your ability to do the real work you'd do at Array: build production-grade software efficiently. Our goal is to give you the opportunity to show how well you can work independently to assimilate requirements and deliver a real-world application across two focused days.

## How it Works

### Rules & Logistics

- You may complete the challenge over any two consecutive days based on your contract start date (coordinate dates with your recruiter).
- Once started, both days must be completed back-to-back.
- We recommend reviewing this document completely in advance so you can ask any clarifying questions before starting.
- You are encouraged to use your favorite tools - including your favorite editor and AI tooling like Cursor and Claude Code. That's how we develop - so you should too.
- Email questions to [eng.interview@array.com](mailto:eng.interview@array.com).

### Deliverables Submission

- Send us your git repo, zipped up and named like this:  
    firstname\_lastname\_engineering\_interview.zip
- Upload it to Google Drive and share a link that will work for any recipient to:  
    [eng.interview@array.com](mailto:eng.interview@array.com).

### How We Evaluate your Submission

We want you to succeed. We focus on:

- **Does it satisfy the requirements?** First and foremost, we need to deliver for our customers - so getting it right matters.
- **Do you make effective use of the Go ecosystem?** There's no point in hand (or AI!) coding a capability built into a mainstream library.
- **How well structured and clear is your implementation?** Code spends more time being maintained and extended than created. Whether you use AI, copy

from the Internet, or hand craft it yourself - you own your code and it needs to be easy to reason about.

- **Is it ready for Production?** Could we run this reliably in production and support it under load? Will it safely cluster and scale horizontally? Show us you can build systems and then go on vacation with confidence.

Plus, if we can't follow your documentation to run and test your work - nothing else matters.

## Your Task - Integrate with NorthWind

No application is an island. Show us how you can connect systems together - reliably - over the Internet.

We've created our own Digital Banking Provider - NorthWind Bank. We want you to integrate an existing banking solution with NorthWind. We've created a baseline application for you - **you'll receive a link to download the git repo at the start of day one.**

Take some time to understand that application and have it work locally, then extend it to integrate with NorthWind.

### Features

- Authentication
  - Not just anyone can transfer funds or enquire into NorthWind. You'll need to implement our authentication scheme using the keys we provide.
- Transfer Funds
  - Your customers want to send money to NorthWind Bank customers. They'll need to be able to register external accounts to transfer to.
  - When they want to do a transfer, you'll need to initiate it and then monitor it for completion. Different transfer types take different amounts of time, so be aware and offer them options.
  - You'll need to be sure their bank account reflects the status of those transfers - including if they eventually fail.
- Notify Regulators
  - Your bank partners with a regulator. For compliance, you must notify it via webhook whenever a transfer succeeds or fails — within 60 seconds. Of course, your regulator may not be up or continuously available, and you should be ready for that.

- You might get audited, so you should keep proof in your database that you complied with your regulator's requirements.

## Deliverables

Send us a (zipped up) copy of your final git repo that includes:

- Go (Golang) application.
- Dockerfile to build and run the app. Don't forget to spin up Postgres as well!
- Postgres database schema.
- Postman collection to demonstrate your exceptional API.